Storm Intensity Distribution — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 01-April-2026

David K Kelly

CONFIDENTIAL — PROPRIETARY

1 Test Results — Storm Intensity Distribution (MKM-SI-001)

Tests run on **01 April 2026 at 10:24:44**.

Table 1: Test Results Summary — Storm Intensity Distribution (MKM-SI-001)

Total Tests	63
Passed	63
Failed	0
Skipped	0
Duration	0.01s

Table 2: Detailed Test Results — Storm Intensity Distribution

Test Description	Acceptance Criteria	Result	Time
All categories defined	`set(DURATION_PARAMS.keys()) == expected`	Pass	0.000s
Catastrophic extended	`high == 240`	Pass	0.000s
Each tier's max roughly doubles the previous.	`1.5 <= ratio <= 3.0`	Pass	0.000s
Extreme extended	`DURATION_PARAMS["extreme"] == (36, 72, 144)`	Pass	0.000s
Moderate extended	`low == 12`; `mode == 30` (+1 more)	Pass	0.000s
Severe extended	`DURATION_PARAMS["severe"] == (24, 48, 96)`	Pass	0.000s
Gap types	`GapType.SHORT.value == "short"`; `GapType.MEDIUM.value == "medium"` (+1 more)	Pass	0.000s
Sequence gap mapping	`SEQUENCE_GAP_TYPE[SequenceType.DOUBLET] == GapType.MEDIUM`; `SEQUENCE_GAP_TYPE[SequenceType.CLUSTER] == GapType.MEDIUM` (+2 more)	Pass	0.000s
Sequence types	`SequenceType.ISOLATED.value == "isolated"`; `SequenceType.DOUBLET.value == "doublet"` (+2 more)	Pass	0.000s
Storm count ranges	`SEQUENCE_TYPE_STORM_COUNTS[SequenceType.ISOLATED] == (1, 1)`; `SEQUENCE_TYPE_STORM_COUNTS[SequenceType.DOUBLET] == (2, 2)` (+2 more)	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Constants	`EVENT_WINDOW_HOURS == 168`; `MIN_DRAINAGE_WINDOW_HOURS == 12` (+1 more)	Pass	0.000s
Exact boundary	`fits_in_window([78.0, 78.0], [0.0])`	Pass	0.000s
Exceeds window	`not fits_in_window([72.0, 72.0], [48.0])`	Pass	0.000s
Simple doublet fits	`fits_in_window([24.0, 24.0], [48.0])`	Pass	0.000s
Single storm	`fits_in_window([100.0], [])`	Pass	0.000s
Long gaps	`GAP_PARAMS[GapType.LONG] == (48, 96, 144)`	Pass	0.000s
Medium gaps	`GAP_PARAMS[GapType.MEDIUM] == (24, 48, 72)`	Pass	0.000s
Short gaps	`GAP_PARAMS[GapType.SHORT] == (6, 18, 36)`	Pass	0.000s
Doublet medium	`get_gap_range(SequenceType.DOUBLET) == (24, 48, 72)`	Pass	0.000s
Isolated zero	`get_gap_range(SequenceType.ISOLATED) == (0, 0, 0)`	Pass	0.000s
Get duration range	`get_duration_range("moderate") == (12, 30, 60)`	Pass	0.000s
Get max duration	`get_max_duration("catastrophic") == 240`; `get_max_duration("minimal") == 12`	Pass	0.000s
Ids unique	`len(ids) == 100`	Pass	0.000s
Sequence id format	`sid.startswith("STORM-")`; `len(sid) == 14`	Pass	0.000s
Sequence ids unique	`len(ids) == 100`	Pass	0.000s
Storm id format	`sid.startswith("STORM-")`; `len(sid) == 14`	Pass	0.000s
Invalid category	—	Pass	0.000s
Mode should be near the most frequent value in a large sample.	`abs(median - mode) < 15`	Pass	0.005s
Line 50: rng=None → creates np.random.RandomState() internally.	`low <= d <= high`	Pass	0.000s
Reproducibility	`d1 == d2`	Pass	0.000s
All sampled durations fall within [min, max] for the category. [category='baseline']	`low <= d <= high`	Pass	0.000s
All sampled durations fall within [min, max] for the category. [category='catastrophic']	`low <= d <= high`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
All sampled durations fall within [min, max] for the category. [category='extreme']	`low <= d <= high`	Pass	0.000s
All sampled durations fall within [min, max] for the category. [category='minimal']	`low <= d <= high`	Pass	0.000s
All sampled durations fall within [min, max] for the category. [category='moderate']	`low <= d <= high`	Pass	0.000s
All sampled durations fall within [min, max] for the category. [category='severe']	`low <= d <= high`	Pass	0.000s
Doublet gaps should fall in medium range (24-72h).	`all(24 <= g <= 72 for g in gaps)`	Pass	0.000s
Isolated returns zero	`sample_gap(SequenceType.ISOLATED) == 0.0`	Pass	0.000s
Line 49: rng=None → creates np.random.RandomState() internally.	`low <= g <= high`	Pass	0.000s
Persistent gaps should fall in short range (6-36h).	`all(6 <= g <= 36 for g in gaps)`	Pass	0.000s
Reproducibility	`g1 == g2`	Pass	0.000s
Within bounds[sequencetype.cluster] [seq_type=¡SequenceType.CLUSTER: 'cluster'¿]	`low <= g <= high`	Pass	0.000s
Within bounds[sequencetype.doublet] [seq_type=¡SequenceType.DOUBLET: 'doublet'¿]	`low <= g <= high`	Pass	0.000s
Within bounds[sequencetype.persistent] [seq_type=¡SequenceType.PERSISTENT: 'persistent'¿]	`low <= g <= high`	Pass	0.000s
Create basic	`storm.storm_id == "STORM-abc12345"; storm.duration_hours == 24.0`	Pass	0.000s
From dict round trip	`restored.storm_id == original.storm_id; restored.duration_hours == original.duration_hours` (+6 more)	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Precipitation rounding	`d["precipitation_mm"] == 35.12`; `d["intensity_factor"] == 1.234568` (+1 more)	Pass	0.000s
To dict	`d["storm_id"] == "STORM-test1234"`; `d["start_time_hours"] == 48.0` (+6 more)	Pass	0.000s
Antecedent defaults	`seq.antecedent_soil_moisture == "normal"`; `seq.antecedent_groundwater == "normal"`	Pass	0.000s
Create doublet	`seq.num_storms == 2`; `len(seq.storms) == 2` (+2 more)	Pass	0.000s
From dict round trip	`restored.storm_id == original.storm_id`; `restored.duration_hours == original.duration_hours` (+6 more)	Pass	0.000s
Isolated no gaps	`len(seq.inter_storm_gaps_hours) == 0`	Pass	0.000s
Spatial correlation defaults	`seq.spatial_correlation_enabled is False`; `seq.spatial_correlation_range_km is None`	Pass	0.000s
Spatial correlation enabled	`d["spatial_correlation_enabled"] is True`; `d["spatial_correlation_range_km"] == 43.2`	Pass	0.000s
To dict	`d["storm_id"] == "STORM-test1234"`; `d["start_time_hours"] == 48.0` (+6 more)	Pass	0.000s
Duration mismatch	`any("total_duration_hours" in e for e in errors)`	Pass	0.000s
Exceeds precip window	`any("MAX_PRECIP_END_HOUR" in e for e in errors)`	Pass	0.000s
Invalid intensity category	`any("invalid intensity" in e for e in errors)`	Pass	0.000s
Invalid sequence type	`any("Invalid sequence_type" in e for e in errors)`	Pass	0.000s
Overlapping storms	`any("starts at" in e and "before storm" in e for e in err...`	Pass	0.000s
Storm count mismatch	`any("num_storms" in e for e in errors)`	Pass	0.000s
Too many storms	`any("not in [1, 5]" in e for e in errors)`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Valid doublet	`errors == []`	Pass	0.000s

1.1 Test Document History

Date	Version	Description	Author
05-Mar-2026	1.0	Initial test suite (12 tests)	D. Kelly
07-Mar-2026	1.1	73 test(s) added; 12 test(s) removed (total: 146)	D. Kelly
08-Mar-2026	1.2	42 test(s) added (total: 115)	D. Kelly
12-Mar-2026	1.3	2 test(s) added (total: 117)	D. Kelly
27-Mar-2026	1.4	12 test(s) removed (total: 105)	D. Kelly
01-Apr-2026	1.5	42 test(s) removed (total: 63)	D. Kelly